DOCKER

Hands-On Lab

What is Docker?

C

Containerization

Package your app with all its dependencies

W

Solves 'Works on My Machine'

Same environment in dev, test, and production

L

Lightweight

Unlike VMs - shares the host OS

F

Fast Deployment

Run in seconds instead of minutes

Virtual Machine vs Docker

What's the difference?

Virtual Machine	Docker Container
Size Heavy - Multiple GBs	**Size** Light - Just MBs
Startup Minutes	**Startup** Seconds
Resources Full OS per VM	**Resources** Shares host OS
Isolation Complete - Separate OS	**Isolation** Process-level
Portability Low	**Portability** Very High

Core Docker Concepts

1

Image

Read-only template
Contains code, libraries, dependencies

2

Container

Running instance of an Image
Isolated runtime environment

3

Dockerfile

Instructions file
To build an Image

Why Docker After API Course?

The Problem

API works only 'on my machine'
Different setups between developers
Difficult to deploy to production

The Solution

Same environment everywhere
Automatic dependency installation
Simple and fast deployment

Faster Development
Start working immediately without complex installations

Better Collaboration
Whole team works in the same environment

Reliable Testing
Same environment = no surprises in staging

Your First Dockerfile

05

```
FROM node:18

WORKDIR /app

COPY package*.json ./
RUN npm install

COPY . .

EXPOSE 3000

CMD ["npm", "start"]
```

- Base image (Node.js 18)
- Working directory
- Install dependencies
- Copy code
- Expose port
- Run command

Build and Run Container

1 Build the Image

```
docker build -t my-api .
```

2 Run the Container

```
docker run -p 3000:3000 my-api
```

Your app is running!

Go to http://localhost:3000

Docker Compose - Multiple Services

API + Database + Redis in one system

```yaml
version: '3.8'
services:
  api:
    build: .
    ports: ["3000:3000"]
  db:
    image: postgres:15
  cache:
    image: redis:7
```

Useful Docker Commands

L

`docker ps`
List running containers

I

`docker images`
List images

V

`docker logs <n>`
View logs

E

`docker exec -it <n> bash`
Enter container shell

S

`docker stop <n>`
Stop container

R

`docker rm <n>`
Remove container

Hands-On Exercise - Your Task

Turn your API from the previous course into Docker

1. **Create Dockerfile**
   In your project directory, create a Dockerfile

2. **Choose base image**
   FROM node:18 or python:3.11 based on your language

3. **Set up environment**
   WORKDIR, COPY, RUN npm install

4. **Build the Image**
   docker build -t my-project-api .

5. **Run Container**
   docker run -p 3000:3000 my-project-api

6. **Test it works**
   Go to localhost:3000 and verify API responds

Best Practices

D **Use .dockerignore**
Don't include node_modules, .git, test files

L **Efficient layers**
Copy package.json before code - leverage cache

S **Small base images**
node:18-alpine instead of node:18 - saves space

E **Environment variables**
Use ENV or --env-file for configuration

M **Multi-stage builds**
Separate build and production images

R **Don't run as root**
Create separate user - better security

Common Errors & Solutions

Port already in use

Solution: docker ps to find the container and stop it

Image build fails

Solution: Check paths in COPY and .dockerignore file

Container exits immediately

Solution: docker logs <n> to see the error

Can't connect to DB

Solution: Use docker-compose networks or service name

Additional Learning Resources

D

Official Documentation

docs.docker.com

Comprehensive guides and API reference

H

Docker Hub

hub.docker.com

Image repository - find ready-made images

P

Play with Docker

labs.play-with-docker.com

Practice in browser without installation

C

Docker Compose Docs

docs.docker.com/compose

Complete guide for multi-container apps